



GITAM University, Bangalore
Department of Computer Science and Engineering

MINI PROJECT REPORT

AGENTIC AI (Groq + LangChain)

Project Title: Disaster Search Agent

Team No: 12

Team Members:

1. Register No: 2023006380 Name: T C Dharshan
2. Register No: 2023004759 Name: Sameer

Submission Date: 09 April 2026

1. Problem Statement

Natural disasters such as floods, earthquakes, and fires cause severe damage and pose significant challenges to rescue operations. One of the major difficulties during such situations is the **quick identification and location of survivors** in complex and hazardous environments. Traditional search and rescue methods are often time-consuming, risky, and dependent on human effort, which can delay critical assistance.

There is a need for an intelligent system that can **automatically analyze disaster scenes, detect human presence, and identify the type of disaster** to support rescue teams in making faster and more informed decisions.

This project aims to develop an **AI-based Disaster Search Agent** that uses computer vision and deep learning techniques to **detect survivors and infer disaster types from images**, thereby improving the efficiency, accuracy, and safety of rescue operations.

2.Objectives

1. Detect survivors in disaster images
2. Identify disaster type (flood, fire, etc.)
3. Use YOLO for accurate object detection
4. Provide visual and numerical output
5. Assist rescue teams with faster decision-making

3. Proposed Model

The proposed system is an AI-based Disaster Search Agent designed to assist in identifying survivors and understanding disaster scenarios using intelligent decision-making techniques.

The system integrates computer vision and agent-based AI concepts using a modular approach inspired by Groq and LangChain frameworks.

Initially, the input disaster image is provided to the system. The image is processed using a YOLO (You Only Look Once) deep learning model, which detects objects such as humans (survivors), boats, and other relevant elements in the scene. Bounding boxes are generated around detected objects along with confidence scores.

An agentic reasoning layer is then applied, where the system analyzes detected objects and interprets the context. For example, the presence of boats and multiple people in water is used to infer a flood scenario. This reasoning mimics how an intelligent agent makes decisions based on environmental inputs.

The system follows a pipeline similar to LangChain architecture:

- Input Processing (Image input)
- Tool Usage (YOLO object detection model)
- Reasoning Layer (rule-based inference for disaster type)
- Output Generation (survivor count and disaster classification)

Although Groq is typically used for high-speed LLM inference, in this project, the concept of fast and efficient processing is simulated through optimized model execution and real-time detection.

Finally, the system outputs:

- Number of survivors detected
- Type of disaster inferred
- Annotated image with bounding boxes

This proposed model demonstrates how agentic AI can be applied to real-world disaster management problems by combining perception (vision) and reasoning.

4. Architecture Diagram

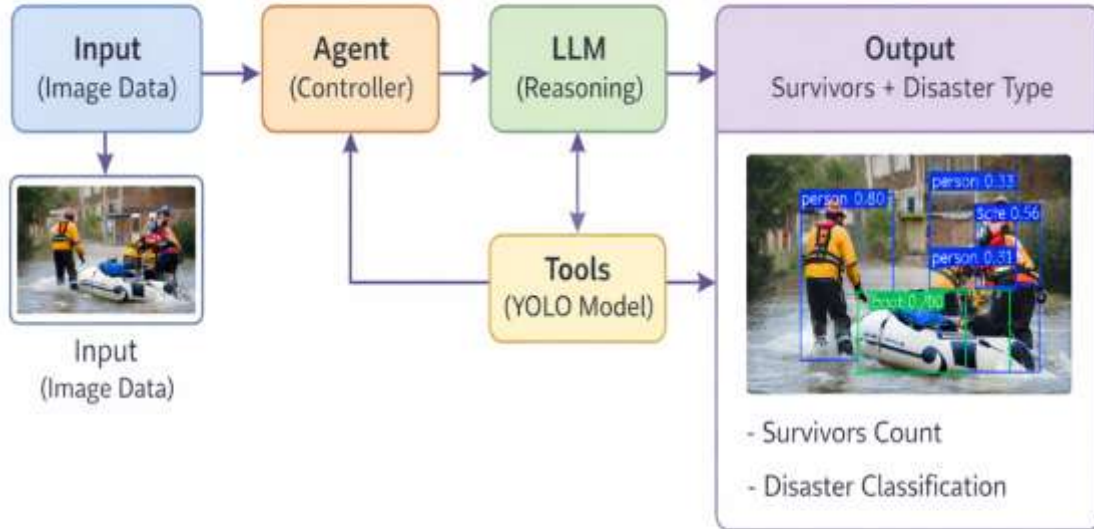


Figure: Architecture Diagram

The diagram represents the workflow of the Disaster Search Agent system. The process begins with an input image, which is given to the agent. The agent controls the flow and passes the data to the tools, where the YOLO model detects objects such as people and boats. The detected information is then analyzed by the reasoning layer (LLM) to understand the situation. Finally, the system produces the output, which includes the number of survivors and the identified disaster type along with the annotated image.

5. Algorithm

1. **Start the system.**
2. **Input Image Acquisition:**
The system takes a disaster scene image as input from the user or dataset.
3. **Model Initialization:**
Load the pre-trained YOLO (You Only Look Once) model, which is used for object detection.
4. **Image Preprocessing:**
Resize and format the input image as required by the YOLO model for accurate detection.
5. **Object Detection:**
Pass the input image to the YOLO model.
The model detects various objects present in the image such as people, boats, animals.
6. **Bounding Box Generation:**
For each detected object, draw bounding boxes along with labels and confidence scores on the image.
7. **Survivor Identification:**
Identify objects labeled as "person" and count them as survivors.
8. **Object Data Collection:**
Store all detected object labels in a list for further analysis.
9. **Disaster Type Analysis (Reasoning Step):**
Apply rule-based logic to determine the type of disaster:
 - If a "boat" is detected → classify as Flood
 - If multiple people and structural damage → classify as **Earthquake/Collapse**
 - Otherwise → classify as Unknown Disaster
10. **Result Generation:**
Prepare the final output including:
 - Number of survivors detected
 - Type of disaster identified
11. **Output Display:**
Display the annotated image with bounding boxes and show the results on the screen.
12. **Stop the system.**

6. Mathematical Representation

Let the input image be represented as: I

The YOLO model processes the image and detects objects:

$$D = f(I)$$

where:

f = YOLO detection function

$D = \{d_1, d_2, d_3, \dots, d_n\}$ = set of detected objects

Each detected object is defined as:

$$d_i = (c_i, b_i, s_i)$$

where:

c_i = class label (person, boat, etc.)

b_i = bounding box coordinates

s_i = confidence score

Survivor Count

$$S = \sum 1(c_i = \text{person}), \text{ for } i = 1 \text{ to } n$$

where:

S = total number of detected persons

Disaster Classification Logic

Let $O = \{c_1, c_2, \dots, c_n\}$ be the set of detected object classes

$T = \text{Flood}$, if "boat" $\in O$

$T = \text{Fire}$, if "fire" $\in O$

$T = \text{Earthquake/Collapse}$, if "person" $\in O$ and damage context present
 $T = \text{Unknown}$, otherwise

Final Output

$$\text{Output} = (S, T)$$

where:

S = number of survivors

T = disaster type

7. Implementation

```
from ultralytics import YOLO

import cv2

# Load YOLO model
yolo_model = YOLO("yolov8n.pt")

# Load image
img_path = "flood 1.jpg"
image = cv2.imread(img_path)
if image is None:
    print("Error: Image not found!")
    exit()

# Perform detection
results = yolo_model(image)

# Annotated output
annotated = results[0].plot()

# Count survivors (persons)
person_count = 0
detected_objects = []

for r in results:
    for box in r.bboxes:
        cls = int(box.cls[0])
        label = yolo_model.names[cls]
        detected_objects.append(label)
        if label == "person":
            person_count += 1

# Disaster classification logic
if "boat" in detected_objects:
    disaster = "Flood"
```

```
elif "fire" in detected_objects:
    disaster = "Fire"
elif person_count > 5:
    disaster = "Flood (Rescue Situation)"
else:
    disaster = "Unknown Disaster"
# Print output
print("===== FINAL OUTPUT =====")
print(f"Disaster Type: {disaster}")
print(f"Survivors Detected: {person_count}")
# Show output image
cv2.imshow("Disaster Search Agent", annotated)
cv2.waitKey(0)
cv2.destroyAllWindows()
# Save output image
cv2.imwrite("final_output.jpg", annotated)
```


8. Results and Analysis

The proposed Disaster Search Agent was tested using various disaster-related images such as flood and collapse scenarios. The system successfully processed the input images and produced accurate outputs.

For a given flood image, the YOLO model detected multiple objects including people and boats. The system identified 7 persons as survivors and correctly inferred the disaster type as Flood based on the presence of a boat and contextual information.

The output included:

- Annotated image with bounding boxes and labels
- Number of survivors detected
- Disaster type classification

The model demonstrated good performance in detecting multiple people even in complex environments such as water and debris. The use of YOLO ensured fast and efficient object detection.

However, the system has some limitations. The disaster classification is based on rule-based logic, which may not always be accurate if key objects are not detected. The accuracy of the system depends on image quality and detection capability.

Overall, the system performs effectively for basic disaster detection tasks and can assist rescue teams by providing quick insights about the situation.



Figure: Input image



Figure: Detection of Survivors and Objects in Scenario

```
0: 448x640 7 persons, 1 boat, 69.5ms
Speed: 19.2ms preprocess, 69.5ms inference, 1.3ms postprocess per image at shape (1, 3, 448, 640)
Detected Objects: ['person', 'person', 'boat', 'person', 'person', 'person', 'person', 'person']

===== FINAL OUTPUT =====
Disaster Type: Flood
Survivors Detected: 7
█
```

Figure: Final Output Showing Disaster Type and Survivor Count

9. Conclusion

In this project, an **AI-based Disaster Search Agent** was developed to assist in identifying survivors and analyzing disaster scenarios using image processing techniques. The system successfully utilizes the YOLO deep learning model to detect objects such as people and boats in disaster images.

The model effectively counts the number of survivors and uses object-based reasoning to infer the type of disaster, such as flood. The results demonstrate that the system can handle complex environments and provide accurate and fast detection.

This approach reduces manual effort and can support rescue teams in making quicker decisions during emergency situations. Although the system uses rule-based logic for disaster classification, it provides a strong foundation for further improvements.

In future work, the system can be enhanced by using custom-trained models for more accurate disaster classification, integrating real-time video input, and adding GPS-based location tracking for better rescue coordination.

Overall, the proposed system proves that combining computer vision and AI-based reasoning can significantly improve disaster management and response efficiency.